inMediaVeritas

Project Documentation

Implementation and technical documentation. For the theory, cryptographic background, and technology behind the project — explored in greater depth — see the paper
inMediaVeritas: A Decentralized News Verification Application Using zk-SNARKs and Blockchain Consensus *(2026)*.

This document describes the **actual implementation** of the proof-of-concept: the components that were built, how they fit together, how to run them, and which parts of the design from the paper are implemented versus planned. The paper should be read for the conceptual background, the threat model, and the formal evaluation (gas costs, zk-SNARK timing). This document is the practical, code-level counterpart.

Live landing page & demo: `https://jakubivacek.github.io/inMediaVeritas/`
The landing page contains a project overview, the three-step workflow (Reporters → Users → Agencies), and a *"See It In Action"* video walkthrough of the working proof-of-concept (role selection and sign-in, zk-SNARK identity proof generation, event capture, verification submission, and voting).

Contents

1 Introduction 1

2 System overview 3

3 Repository layout 4

4 Component reference 4
4.1 Smart contract — `media-voting-smart-contract` . . . 4
4.2 Backend API & infrastructure — `media-voting-backend` . . . 6
4.3 Main application — `IMV_FullApp` . . . 7
4.4 Identity wallet — `ID_app` . . . 8
4.5 Authority portal — `AUTHORITY` . . . 9
4.6 Zero-knowledge circuits — `ZK-SNARKs` . . . 9

5 End-to-end flow 10

6 Cryptographic design 11

7 Technology stack 12

8 Build & run instructions 12

9 Implementation status 13

10 Limitations and future work 14

11 Glossary 15

1 Introduction

inMediaVeritas is a decentralized application (DApp) that aims to combat misinformation by combining three ideas:

- **Cryptographic provenance** — media is hashed and its content identifier (CID) is committed on-chain, so any later tampering is detectable.

- **Privacy-preserving community verification** — geographically and identity-eligible users vote on the credibility of reported media, proving their eligibility with **zk-SNARKs** without revealing who they are or exactly where they are.
- **No central arbiter of truth** — credibility is established by on-chain consensus rather than by an editorial board.

The system is organized around three actor tiers, mirrored in the application's user roles:

Tier	Role in the apps	What they do
Reporter ("roaming / journalist")	`(journalist)`, `(newsAgency)`	Capture media, attach a location, publish an event on-chain.
Witness ("local user")	`(localUser)`	Confirm or deny an event by voting, proving local eligibility with a zk-SNARK.
Reader / News Agency	`(reader)`, `(newsAgency)`	Browse verified events and their vote breakdown.

This documentation focuses on what exists in the repository today: a locally-deployed smart contract, a read-only backend with IPFS storage, the main voting/reporting mobile app, a separate identity-wallet mobile app, the credential-issuing authority portal, and the Circom circuits with their proving pipeline.

This directory is **not a single git repository.** Each of the six component directories is its own independent git repository, while the parent directory itself is not under version control:

- `media-voting-smart-contract/`
- `media-voting-backend/`
- `IMV_FullApp/`
- `ID_app/`
- `AUTHORITY/`
- `ZK-SNARKs/`

There is no top-level build or workspace that ties everything together; each component is cloned, built, and run on its own.

How to read this document. Section 2 presents the system-level architecture and the trust flow that links the components. Section 3 lists the repository layout, and Section 4 is the detailed per-component reference — the smart contract, the backend, the main app, the identity wallet, the authority portal, and the zero-knowledge circuits. Section 5 walks the complete end-to-end flow from onboarding to a counted vote, and Section 6 collects the cryptographic design in one place. Sections 7–8 cover the technology stack and how to build and run each component. Section 9 gives an honest implemented-versus-planned status, Section 10 the limitations and future work, and Section 11 a glossary of recurring terms.

2 System overview

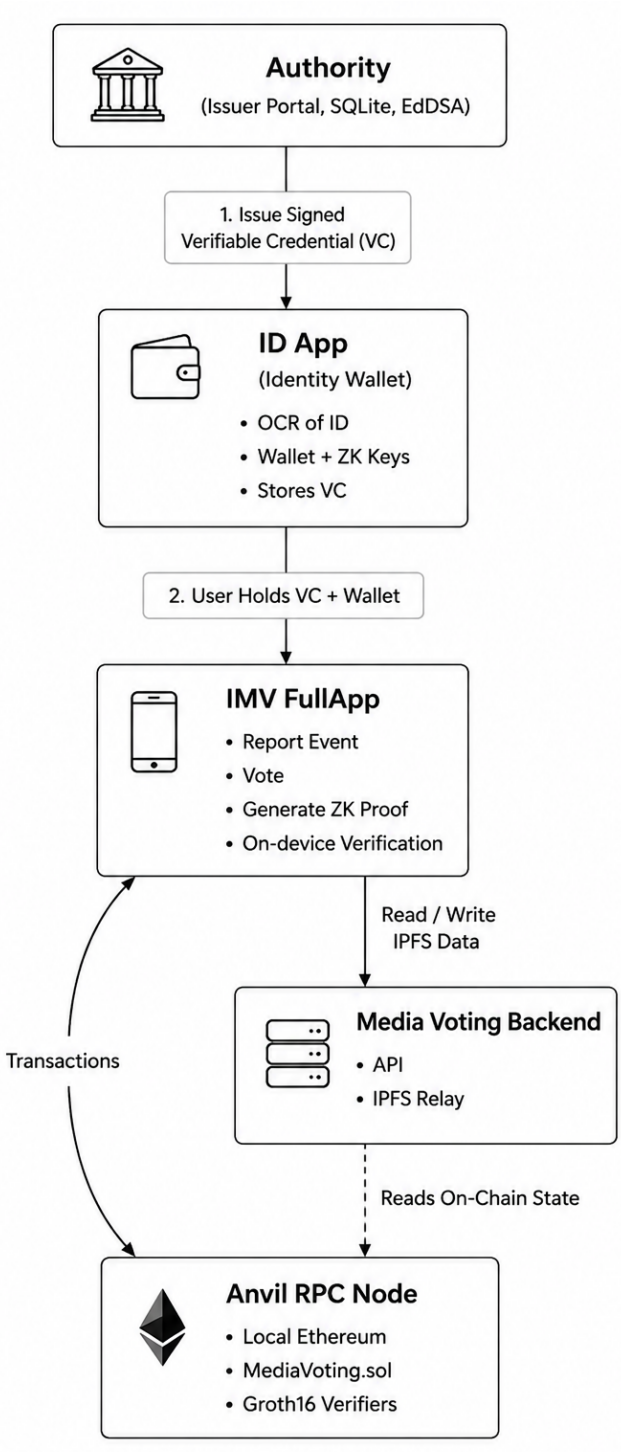


Figure 1: Architecture overview: the trust flow linking the authority, the identity wallet, the main app, the backend relay, and the on-chain layer.

The **trust flow** that links the components:

1. **AUTHORITY** verifies a person's real-world identity (an official reviews ID images) and issues a **Verifiable Credential (VC)** signed with the authority's EdDSA key over the Baby JubJub curve.

2. **ID_app** captures the ID document with the camera, generates a blockchain wallet (secp256k1) and ZK keys, requests the VC from the authority, and stores it on the device. (Credential fields are entered manually; automatic OCR text recognition is scaffolded but not yet wired — see Section 4.4.)

3. To vote, the user (in **IMV_FullApp**) generates a **zk-SNARK proof** on-device proving they hold a valid VC (and, for the location circuit, that the VC's locality matches the event's required region) — **without revealing the VC's contents**.

4. **IMV_FullApp** uploads media to **IPFS** through the backend, then signs and broadcasts the `createEvent` / `voteOnMedia` / `voteOnMediaWithProof` transactions **directly from the mobile wallet to the Anvil RPC**.

5. **MediaVoting.sol** stores event metadata and media references on-chain, deduplicates votes by a `bytes32` VC-derived hash, tracks global vs. local vote tallies, and (in ZK mode) verifies the Groth16 proofs before counting a vote.

6. **media-voting-backend** is **read-only** with respect to the chain: it serves on-chain event/media state over a REST API and relays IPFS uploads/reads. It never signs a chain-state-changing transaction.

Key architectural invariant.

The **mobile wallet signs all chain writes**; the backend only reads the chain and relays IPFS. Because IPFS CIDs are self-verifying content hashes and all consensus state lives on-chain, the backend is a convenience relay, not a trust anchor — it can neither forge media nor influence vote outcomes. In plain terms: even if the backend were compromised or malicious, it could not cheat — user wallets sign every action, each file's CID lets anyone verify it was not tampered with, and all votes live on-chain, so the backend could be swapped for a public IPFS gateway without weakening the system's security.

3 Repository layout

4 Component reference

4.1 Smart contract — `media-voting-smart-contract`

The on-chain core is a single Solidity contract, `MediaVoting.sol`, built and tested with **Foundry**. The optimizer and `via_ir` are enabled (see `foundry.toml`).

Data model.

A `NewsEvent` aggregates one or more `MediaItems`. Voting happens **per media item**, not per event (an event-level vote also exists). Each event and each media item keeps four counters — `yesCount`, `noCount`, `localYesCount`, `localNoCount` — and a `mapping(bytes32 => bool)` that deduplicates voters by their VC-derived hash.

```
NewsEvent
 |- id, name, description, location, timestamp
 |- reporter (address), reporterName, reporterOrg
 |- yes / no / localYes / localNo counters
 |- hasVotedOnEvent: mapping(bytes32 => bool)
 \- MediaItem[]
        |- id, uri (IPFS CID), description, mediaType
        |- yes / no / localYes / localNo counters
        \- hasVoted: mapping(bytes32 => bool)
```

Directory	Role	Primary stack
`media-voting-smart-contract/`	`MediaVoting` contract: on-chain events, media items, votes, ZK verifier hooks	Solidity + Foundry
`media-voting-backend/`	Read API for on-chain events + IPFS upload/gateway relay; local infra (Anvil, IPFS) via Docker Compose	TypeScript, Express, ethers v6
`IMV_FullApp/`	Main voting/reporting app — npm-workspaces monorepo: `IMV_app` (UI) + `IMV_ClientLogic` + `packages/shared-types`	Expo / React Native
`ID_app/`	Identity-credential wallet app (camera ID capture + manual entry, wallet + ZK key generation, holds the VC)	Expo / React Native
`AUTHORITY/`	Issuer/official portal that attests users and issues Verifiable Credentials	Node.js + Express, Vue 3 + Vite, SQLite
`ZK-SNARKs/`	Circom circuits: `identity-verifier` and `location-verifier`	Circom + snarkjs (Groth16)

Event IDs equal the index in the `newsEvents` array; media IDs equal the index in the event's `mediaItems` array.

Public interface (the integration contract).

Writes

```
createEvent(name, description, location, timestamp,
            reporterName, reporterOrg, MediaInput[]) -> eventId
addMediaItem(eventId, uri, description, mediaType)  -> mediaId
voteOnEvent(eventId, voterHash, support, isLocalVoter)
voteOnMedia(eventId, mediaId, voterHash, support, isLocalVoter)
voteOnMediaWithProof(eventId, mediaId, voterHash, support,
                     voteProofData, vcHashProofData)
```

- `createEvent` — creates an event and optionally appends media items in the same transaction.
- `addMediaItem` — appends a media item to an existing event.
- `voteOnEvent` — event-level vote (demo mode only).
- `voteOnMedia` — **demo / non-ZK** media vote; requires `zkVotingEnabled == false`.
- `voteOnMediaWithProof` — **ZK mode** media vote; requires `zkVotingEnabled == true` and a configured verifier. Delegates proof verification to the verifier contract and derives `isLocalVoter` from the proof.

Reads (view)

- `getEvent(eventId)`, `getMediaItem(eventId, mediaId)`, `getEventCount()`, `getMediaCountForEvent(eventId)`
- `hasVotedOnEvent(eventId, voterHash)`, `hasVotedOnMedia(eventId, mediaId, voterHash)`

Admin (owner)

- `setZKVerifier(address)` — sets the verifier adapter.

- `setZKMode(zkVotingEnabled, zkVCHashCheckEnabled)` — toggles between demo voting and proof-based voting, and whether the VC-hash proof is additionally required.

Events: `EventCreated`, `MediaItemAdded`, `EventVoted`, `MediaVoted`, `ZKVerifierUpdated`, `ZKModeUpdated`.

ZK gating.

The contract has two voting paths. In **demo mode** (`zkVotingEnabled == false`) the client supplies `voterHash` and `isLocalVoter` directly via `voteOnMedia`. In **ZK mode** (`zkVotingEnabled == true`) only `voteOnMediaWithProof` is accepted; it calls into an `IZKVoteVerifier` implementation:

- `IZKVoteVerifier.sol` — the verifier interface (`verifyVoteProof`, `verifyVCHashProof`).
- `MockZKVoteVerifier.sol` — a mock used in tests, so the voting logic can be exercised without real proofs.
- In a full deployment the adapter (`ZKVoteVerifierAdapter`) forwards to the snarkjs-generated Groth16 verifier contracts (identity + location).

Tests live in `test/MediaVoting.t.sol` and cover event creation, media addition, voting, duplicate-vote prevention, and global-vs-local counting.

ZK voting validated end-to-end.

Beyond the mock, the full ZK path was exercised for the paper: the snarkjs-generated Solidity verifiers for *both* circuits (identity and location) were deployed, and real proofs produced by the proving pipeline were submitted on-chain — both verified successfully. The paper (§VIII) reports the measured on-chain verification cost (~215k–228k gas per proof), confirming that proof-based voting via `voteOnMediaWithProof` works on a real EVM, not just against the mock.

4.2 Backend API & infrastructure — `media-voting-backend`

The backend is **read-only with respect to the chain** and acts as an **IPFS relay**. The active implementation is the TypeScript service in `api/` (ESM, `"type": "module"`). The legacy root `package.json` and a JavaScript `index.js` variant are superseded — all work happens in `api/`.

```
api/src/
  app.ts                              # Express app wiring (CORS, JSON, routes)
  server.ts                           # bootstrap
  config/                            # env + service container
  routes/                            # health, ipfs, events
  controllers/                       # request handlers
  services/
    blockchain/BlockchainService.ts  # ethers v6 read + (optional) signer
    ipfs/IpfsService.ts              # Kubo HTTP client
    event/EventService.ts
  middleware/errorHandler.ts
  types/event.ts
```

REST API.

Health

- `GET /health`
- `GET /health/deep` — checks chain reachability, contract callability, and IPFS health.

IPFS

- `POST /api/ipfs/images` — multipart upload (field name `images`); returns one CID per file. If `IPFS_UPLOAD_API_KEY` is configured, an `x-api-key` header is required.

- `GET /api/ipfs/:cid` — redirects to the file on the IPFS gateway; `?meta=1` returns JSON (`cid`, `gatewayUrl`).

Events (read on-chain state)

- `GET /api/events?limit=&offset=` — list events.
- `GET /api/events/:id` — one event.
- `GET /api/events/:id/media` — media items for one event.

Voting is **not** exposed by the backend — it is submitted directly from the mobile wallet to the RPC.

`BlockchainService`.

Wraps an ethers v6 `JsonRpcProvider` and a `Contract`. It loads the ABI from `CONTRACT_ABI_PATH` (defaults to `./abi/MediaVoting.json`) and falls back to a hard-coded minimal ABI if the file is missing. It reads events/media and maps the contract's `uri` field to `cid` for API compatibility. It also holds a signer constructed from `PRIVATE_KEY`, but the production design keeps writes on the mobile wallet; the read endpoints are the ones used by the apps.

ethers v6 detail.

uint256 values come back as `BigInt`. `app.ts` patches `BigInt.prototype.toJSON` so Express can `res.json()` them.

`IpfsService`.

Uses `ipfs-http-client` against a local **Kubo** node. `uploadFile` adds the buffer and returns `\{ cid, fileName, gatewayUrl \}`. `buildGatewayUrl` constructs `<gatewayBaseUrl>/ipfs/<cid>`. Health is checked via the Kubo `/version` endpoint.

Local infrastructure.

`infra/docker-compose.yml` brings up three services: **anvil** (local Ethereum dev chain, deterministic mnemonic), **ipfs** (local Kubo node, internal only `ipfs:5001`), and **api** (the TypeScript backend, public entrypoint `http://localhost:8000`). PowerShell scripts under `scripts/` automate the lifecycle: `bootstrap.ps1` (full bring-up: infra → deploy → wire `.env` → health → smoke test), `deploy-contract.ps1` / `deploy-contract-wsl.ps1`, `deploy-all.ps1`, and `export-devnet-accounts.mjs`. Foundry/`forge` is expected to run inside **WSL Ubuntu** on this Windows setup.

Configuration.

Comes from `infra/.env` (copy from `infra/.env.example`). Key values: `ANVIL_MNEMONIC`, `PRIVATE_KEY` (deploy only), `CONTRACT_ADDRESS`, `CHAIN_ID` (31337), `ANVIL_RPC_URL`, `IPFS_API_URL`, `IPFS_GATEWAY_BASE_URL`. The exact mobile JSON-RPC ordering for create/vote is documented in `docs/mobile-rpc-integration-runbook.md`.

4.3 Main application — `IMV_FullApp`

The main app is an **Expo / React Native** project organized as an **npm-workspaces monorepo** with a shared `node_modules` at the monorepo root.

```
IMV_FullApp/
  IMV_app/                # the Expo UI (expo-router)
  IMV_ClientLogic/        # React Native client logic (native modules host)
  packages/shared-types/  # shared TypeScript types  (@imv/shared-types)
```

Routing & roles.

`IMV_app` uses **expo-router** with route groups by user role: `app/(journalist)/` (`report_event`, `browse_events`, `reported_event-browse`); `app/(localUser)/` (`browse_events-lu`, `reported_event-lu`); `app/(newsAgency)/` (`subscriptions`, `reported_event-na`); `app/(reader)/`; and top-level `login`, `index`, `generateZkSnark`, `reported_event`, plus deep-link handling. Shared app state lives in `context/AppContext.tsx`; shared types are imported as `@imv/shared-types`.

On-device zk-SNARK proof generation.

This is the most distinctive part of the app. Proofs are generated **client-side on the phone**, using `@iden3/react-native-circom-witnesscalc` (computes the witness from compiled circuit graph operations, `.wcd`) and `@iden3/react-native-rapidsnark` (generates and verifies the Groth16 proof). `app/generateZkSnark.tsx` exposes `generateProof(input, type)`:

1. Looks up the circuit assets for `type` (currently the `identity` circuit: `identity4.wcd`, `identity41.zkey`, `verification_key.json`) from `CIRCUIT_MAP`.
2. Materializes the `.wcd` graph to the cache directory and reads it as base64.
3. `calculateWitness(inputString, graph)` → witness.
4. `groth16Prove(zkeyPath, witness)` → `\{ proof, pub_signals \}`.
5. `groth16Verify(...)` is run immediately as a sanity check — the witness can be computed for invalid inputs, so verification is what actually catches them.

The resulting proof + public signals are submitted on-chain via the proof-based voting path. Media files are uploaded to IPFS through the backend, and the wallet signs/broadcasts the transactions directly. Per the paper, on-device proving completes in well under 300 ms on recent phones.

4.4 Identity wallet — `ID_app`

A separate **Expo / React Native** app that plays the role of the **identity wallet** (inspired by the EUDI/eIDAS wallet model). It is responsible for onboarding a user, generating their keys, obtaining a VC from the authority, and holding it.

- **ID capture (camera).** An ID-card scanner (`components/id-card-scanner.tsx`, using `expo-camera`) photographs the document, and field-parsing logic (`utils/ocr_utils.ts`) extracts name, dates, address, postal code, and ID number. *Note:* the image-to-text step (`extractTextFromImage`) is currently a stub that returns empty text — it is scaffolded for a cloud OCR service (Google Vision / Azure / AWS) but not yet wired, so in practice the credential fields are entered manually via `id-credential-fill.tsx`.
- **Blockchain wallet generation** (`utils/blockchain_wallet_generation.ts`): 128-bit secure entropy → BIP-39 mnemonic → secp256k1 `ethers` wallet. The private key, public key, and mnemonic are stored in `expo-secure-store`; the address in `AsyncStorage`.
- **ZK key generation** (`utils/zk_generation.ts`): a BLS12-381 keypair (`@noble/curves`), private key in `expo-secure-store`, public key in `AsyncStorage`.
- **VC hashing** (`utils/hashing.ts`): reconstructs the canonical VC, packs UTF-8 bytes into field elements (248-bit chunks), and computes **Poseidon** hashes (`poseidon-lite`) for city (postal code), district (address), and metadata — these mirror exactly what the authority signs, so the on-device hash matches the authority's signature.
- **Biometric-protected actions** — `expo-local-authentication` gates sensitive screens (`biometry-gate.tsx`).

- **Credential UI** — `id-credential-fill.tsx`, `id-credential-display.tsx`; deep links in `app/handle/[action].tsx`.
- **Push notifications** — Firebase Cloud Messaging (`@react-native-firebase/*`) receives credential-status updates from the authority.

4.5 Authority portal — `AUTHORITY`

The **issuer / official portal**: where a trusted authority (e.g. a government body) reviews identity applications and issues signed Verifiable Credentials. It has its own git repository, a **Node.js + Express** backend, a **Vue 3 + Vite** frontend, and a **SQLite** database (`authority.db`).

Backend (`backend/authority.js`).

Cryptography is built on `circomlibjs`: an **EdDSA** keypair over **Baby JubJub**, with **Poseidon** hashing. The authority's secret key is loaded from / generated into `auth_key.json` on startup. A JWT secret is auto-generated into `.env`.

VC signing.

When an official accepts a request, `signRequestVC` reconstructs the credential, computes Poseidon hashes for the address, city (postal code), district (address), and metadata, combines them into a single `vcHash = Poseidon(address, cityHash, districtHash, metaHash)`, and signs that with `eddsa.signPoseidon`. The signed VC returned to the wallet contains the VC body, the signature (`R8x`, `R8y`, `S`), and the authority public key (`Ax`, `Ay`) — exactly the inputs the Circom circuits expect.

API endpoints.

- `POST /issue-vc-request` — applicant submits VC request + ID images (`multer`: `id_image`, `id_image_back`, `applicant_image`); validated with a **Zod** schema, deduplicated, stored as a pending request.
- `POST /login` — official login (bcrypt + JWT, 8 h expiry). A default user is seeded in the DB layer.
- `GET /pending-requests`, `GET /processed-requests` — JWT-protected queues for the official UI.
- `POST /validate-request` — accept (sign + issue VC, notify wallet) or reject (notify wallet).
- `GET /get-vcs/:request_id` — wallet polls for its issued credential.
- Firebase Admin sends FCM push notifications to the ID_app on credential status changes.

The backend is intentionally minimal. The README documents HTTPS on port **3443** with `npm run gen-certs` for certificates; the current `authority.js` also supports a plain-HTTP listener on `PORT` for local development. The frontend (Vue 3 + Vite) runs on port **5173** and lets officials log in, review pending applications with the submitted ID images, and accept/reject — which triggers credential issuance on the backend.

4.6 Zero-knowledge circuits — `ZK-SNARKs`

Two **Circom** circuits (circom 2.1.5), proven with **Groth16** over the **BN254 (alt_bn128)** curve. They use `circomlib`'s `Poseidon` and `EdDSAPoseidonVerifier`. The **Baby JubJub** curve is used only for the EdDSA operations embedded inside the circuits, not as the proving curve.

Identity circuit (`IdentityCircuit.circom`).

Proves that the prover holds a VC validly signed by the authority and bound to their address.

- **Public inputs:** `pk_auth[2]` (authority public key), `address` (user blockchain address), `vc_address` (address inside the VC), `vc_metadata`.
- **Private inputs (witness):** `vc_city`, `vc_district`, `sig_auth_R8[2]`, `sig_auth_S`.
- **Constraints:** (1) reconstruct the VC message `Poseidon(vc_address, vc_city, vc_district, vc_metadata)`; (2) verify the authority's EdDSA-Poseidon signature on that message; (3) enforce `address == vc_address`.

Location circuit (`LocationCircuit.circom`).

Proves the same credential validity **and** that the credential's locality matches a requested filter (city/district), while keeping the exact attributes private — the privacy-preserving "local witness" proof.

- **Public inputs:** `pk_auth[2]`, `address`, `vote` (0/1), `req_city`, `req_district`, `H_VC`.
- **Private inputs:** `vc_address`, `vc_city`, `vc_district`, `vc_metadata`, `sig_auth_R8[2]`, `sig_auth_S`.
- **Constraints:** (1) recompute the Poseidon hash and enforce it equals the public `H_VC`; (2) verify the authority's EdDSA signature over `H_VC`; (3) enforce `address == vc_address`; (4) enforce `vc_city == req_city` and `vc_district == req_district`; (5) enforce `vote` is boolean.

Note.

A user-signature-over-vote check is present but **commented out**: the user's blockchain key is secp256k1, which cannot be verified inside the Baby-JubJub-based EdDSA gadget. Vote integrity is instead provided by the on-chain transaction signature.

Proving pipeline.

There are no npm scripts — the circuits are compiled and proven manually:

```
circom IdentityCircuit.circom --r1cs --wasm --sym --c
node generate_witness.js IdentityCircuit.wasm input.json witness.wtns
# snarkjs: powersoftau -> groth16 setup -> zkey contribute -> export vkey
snarkjs groth16 prove  IdentityCircuit_0001.zkey witness.wtns proof.json public.json
snarkjs groth16 verify verification_key.json public.json proof.json
```

An `input.json` is required before witness generation. For on-chain verification, `snarkjs zkey export solidityverifier` generates the Solidity verifier contract. For **mobile**, the circuit is compiled into the `.wcd` graph format consumed by `circom-witnesscalc`/`rapidsnark`.

5 End-to-end flow

A. Onboarding & credential issuance.

1. User opens **ID_app**, generates a blockchain wallet (secp256k1) and ZK keys (BLS12-381), photographs their ID document, and fills in the credential fields (automatic OCR is not yet wired).
2. ID_app submits an `issue-vc-request` (with ID images) to **AUTHORITY**.
3. An official logs into the **AUTHORITY** Vue frontend, reviews the pending request and images, and accepts it.
4. AUTHORITY's `signRequestVC` builds `vcHash = Poseidon(address, cityHash, districtHash, metaHash)` and signs it with the authority's EdDSA/Baby-JubJub key; the signed VC (body + signature + authority public key) is stored, and ID_app is notified via FCM.
5. ID_app polls `get-vcs/:request_id`, receives the signed VC, and stores it on the device.

B. Reporting an event (reporter role, IMV_FullApp).

6. The reporter captures media, fills in event metadata and location.
7. IMV_app uploads the media to **IPFS** via `POST /api/ipfs/images` and collects the CIDs.
8. The reporter generates an **identity zk-SNARK proof**: tapping "Generate Identity Proof" deep-links to **ID_app**, which returns the circuit inputs, and the app produces a Groth16 identity proof on-device via `generateProof`. (Only the identity circuit is wired into reporting; the location proof is not yet integrated.)
9. The mobile client broadcasts `createEvent(...)` — with the media CIDs as `MediaInput.uri` together with the identity proof and public signals — to the **Anvil RPC**. `MediaVoting.sol` stores the event + media and emits `EventCreated` / `MediaItemAdded`.

C. Witness voting (local-user role, IMV_FullApp).

10. The witness browses events (read via the backend `/api/events`).
11. To vote, IMV_app builds the circuit input from the stored VC and the event's required locality, then calls `generateProof(...)` to produce a Groth16 proof **on-device**, with an immediate `groth16Verify` self-check.
12. **Demo mode:** the wallet calls `voteOnMedia(eventId, mediaId, voterHash, support, isLocalVoter)`. **ZK mode:** the wallet calls `voteOnMediaWithProof(...)`, and `MediaVoting.sol` verifies the proof through the verifier adapter before counting the vote and deriving `isLocalVoter` from it.
13. The contract deduplicates by `voterHash`, updates the global/local tallies, and emits `MediaVoted`.

D. Consumption (reader / agency role).

14. Readers and agencies browse verified events and the per-media vote breakdown through the backend read API; media is fetched from the IPFS gateway by CID.

6 Cryptographic design

Concern	Mechanism	Where
Credential authenticity	EdDSA signature over Poseidon hash, Baby JubJub curve	AUTHORITY signs; circuits verify
Credential privacy	zk-SNARK (Groth16, BN254) proves VC validity without revealing attributes	`ZK-SNARKs/`, on-device
Locality eligibility	Location circuit enforces `vc_city == req_city`, `vc_district == req_district` privately	`LocationCircuit.circom`
Vote deduplication	`voterHash` (`bytes32`) keyed mapping per media item	`MediaVoting.sol`
Vote integrity	On-chain transaction signature (secp256k1 wallet)	mobile wallet → RPC
Media integrity	IPFS CID is the content hash; (paper) SHA-256 committed on-chain	IPFS + contract `uri`
Key custody	Secure Enclave / Keystore via `expo-secure-store`; biometric gate	ID_app

Hash binding consistency.

The same Poseidon-over-packed-bytes construction is implemented in three places and must stay byte-for-byte identical: the authority signer (`authority.js`), the wallet (`ID_app/utils/hashing.ts`), and the circuits. Field packing uses 248-bit chunks of the UTF-8 bit-stream. Any divergence (e.g. JSON canonicalization or which fields are dropped before hashing) breaks signature verification inside the circuit.

7 Technology stack

- **Smart contracts:** Solidity `^0.8.13`, Foundry (`forge`), optimizer + `via_ir`.
- **Zero-knowledge:** Circom 2.1.5, circomlib (Poseidon, EdDSA), Groth16, snarkjs (desktop), `@iden3/react-native-circom-witnesscalc` + `@iden3/react-native-rapidsnark` (mobile). Proving curve BN254; embedded curve Baby JubJub.
- **Backend:** Node.js, TypeScript (ESM), Express 5, ethers **v6**, `ipfs-http-client`, multer; Docker Compose (Anvil + Kubo IPFS + API).
- **Mobile (IMV_app & ID_app):** Expo (SDK 54), React Native 0.81, React 19, expo-router; ethers **v6** (IMV_app) / **v5** (ID_app); `@noble/curves`, `poseidon-lite`, `expo-secure-store`, `expo-local-authentication`, `react-native-vision-camera`, Firebase (ID_app).
- **Authority:** Node.js + Express, Vue 3 + Vite, SQLite, circomlibjs, bcrypt, JWT, Zod, Firebase Admin.
- **Storage:** IPFS (Kubo). **Chain:** local Anvil dev chain (chain ID 31337), targeting an Ethereum-compatible network / L2 in the paper's design.
- **Host environment:** Windows 11 with WSL 2 (Ubuntu) for Foundry; PowerShell automation scripts.

8 Build & run instructions

Each component is built and run independently. A typical local end-to-end bring-up is: backend infra + contracts first, then the apps, then the authority.

Smart contract (Foundry, in WSL Ubuntu).

```
cd media-voting-smart-contract
forge build
forge test                          # all tests
forge test -vv                      # verbose
forge test --match-test <testName>  # single test
```

Backend + local infra.

One-command bring-up (Windows PowerShell, from `media-voting-backend/`):

```
Copy-Item infra/.env.example infra/.env   # then fill ANVIL_MNEMONIC / PRIVATE_KEY
.\scripts\bootstrap.ps1                    # infra + deploy + wire .env + health + smoke
```

Deploy with Foundry in WSL and restart the API, or run the API by itself:

```
.\scripts\deploy-all.ps1 -UseWsl -WslDistro Ubuntu -FoundryProjectPathLinux "/path/to/contract"

cd media-voting-backend/api
npm install
npm run dev      # tsx watch src/server.ts
```

```
npm run check    # tsc --noEmit (typecheck)
npm run build    # tsc -> dist/
npm start        # node dist/server.js
```

Public API at `http://localhost:8000`; RPC for mobile writes at `http://localhost:8545`.

IMV_FullApp (Expo monorepo).

```
cd IMV_FullApp
npm install            # installs all workspaces (shared node_modules)
npm run start:app      # start IMV_app (Expo)
npm run start:logic    # start IMV_ClientLogic
npm run lint           # lint all workspaces
npm run test           # test all workspaces
```

ID_app (Expo).

```
cd ID_app
npm install
npm start              # expo start (also: npm run android / ios / web)
npm run lint
```

AUTHORITY.

```
cd AUTHORITY/backend  && npm install && npm run gen-certs && npm run dev  # 3443 / PORT
cd AUTHORITY/frontend && npm install && npm run dev                       # Vite/Vue 5173
```

ZK-SNARKs.

Manual compile/prove per Section 4.6. For mobile, the `.wcd` graph and `.zkey` are bundled into `IMV_app/assets/circuits/`.

9 Implementation status

The proof-of-concept implements the **core verification pipeline** end-to-end. Several incentive/consensus mechanisms described in the paper are **designed but not implemented** — the documentation is explicit about this so the evaluation is honest.

Capability	Status
`MediaVoting` contract (events, media, per-media voting, dedup, local/global tallies)	Implemented & tested
ZK verifier hooks + mode switch (`setZKVerifier`, `setZKMode`, mock verifier)	Implemented
Identity & location Circom circuits authored + Groth16 proving (desktop & mobile, both benchmarked in paper §VIII)	Implemented
On-device proof generation in the app (`witnesscalc` + `rapidsnark`)	Implemented — **identity circuit only** (the only entry wired in `CIRCUIT_MAP`)
Location proof wired into the mobile app	Not yet — circuit exists and is benchmarked, but not integrated into on-device proving

Capability	Status
IPFS upload/read relay + read-only event API	Implemented
Identity wallet (camera capture + manual entry, wallet + ZK keys, VC storage, biometrics, FCM)	Implemented
Automatic OCR text recognition in the wallet	Stub only (`extractTextFromImage` returns empty; awaiting cloud OCR integration)
Authority portal (review, EdDSA VC issuance, notifications)	Implemented
Demo (non-ZK) voting path wired end-to-end	Working
Full ZK voting path with real on-chain Groth16 verification	Partial: Identity proof path implemented; on-chain verifiers benchmarked (paper §VIII). Location proof and combined ZK voting not yet wired
Incentive pool / subscription economics	Design only
News agency subscription & licensing — mobile UI	Partial: Frontend only — plan selection + active-subscription tracking (expiry, cancel) in the `(newsAgency)` route group, persisted locally in `AsyncStorage`
News agency subscription & licensing — payments, subscription contract, backend licensing	Not implemented (no on-chain payment, no licensing backend)
Media storage	Partial: Single local **Kubo** IPFS node via `IpfsService.uploadFile` (`client.add`); files rely on Kubo's default local pin, no explicit/remote pinning strategy

10 Limitations and future work

These mirror the paper's *Discussion and Limitations* and the realities of the codebase:

- **Sparse witness coverage.** Correctness depends on enough geographically proximate witnesses being present and willing to vote; low-density areas may not reach quorum. Quorum/resolution logic is not yet implemented.
- **Authority centralization risk.** Credential issuance reintroduces some centralization at the identity layer. The design allows multiple parallel authorities (each must support the same ID_app API); federated governance and cross-authority recognition are future work.
- **Backend relay dependency.** IPFS uploads/reads route through the backend for convenience. It is not a trust anchor (CIDs are self-verifying, consensus is on-chain) and could be replaced by a public gateway or in-browser node, but it remains an availability dependency. Durable / remote pinning (e.g. a pinning service or Arweave, as sketched in the paper) is future work.
- **Vote-signature gap in the location circuit.** Binding the vote to the user's key inside the circuit is disabled because the wallet key is secp256k1; vote integrity currently relies on the on-chain transaction signature.

- **Incentive layer.** Rewards, staking, and slashing are designed but not coded; without them, honest-participation incentives are not yet enforced.
- **Scalability & legal.** Single L1/L2 deployment; high-volume news cycles would stress gas and pinning. Decentralized media licensing also raises unresolved legal/regulatory questions.

11 Glossary

VC (Verifiable Credential)
a signed claim about a user issued by the authority and held in the ID_app wallet.

voterHash
a `bytes32` value derived from the VC, used on-chain to deduplicate votes without revealing identity.

zk-SNARK / Groth16
a succinct zero-knowledge proof system; used to prove VC validity and locality without revealing private attributes.

Poseidon
a hash function efficient inside arithmetic circuits.

EdDSA / Baby JubJub
the signature scheme/curve the authority uses, verifiable inside Circom.

BN254 (alt_bn128)
the proving curve, with native Ethereum precompiles for cheap on-chain verification.

CID
IPFS content identifier; the content hash that addresses a stored file.

Anvil
Foundry's local Ethereum development node.

Kubo
the reference Go implementation of an IPFS node.

For conceptual background, the formal threat model, gas-cost tables, and zk-SNARK timing measurements, see the accompanying paper **inMediaVeritas (2026)**. *For a guided demo, see the landing page:* https://jakubivacek.github.io/inMediaVeritas/.